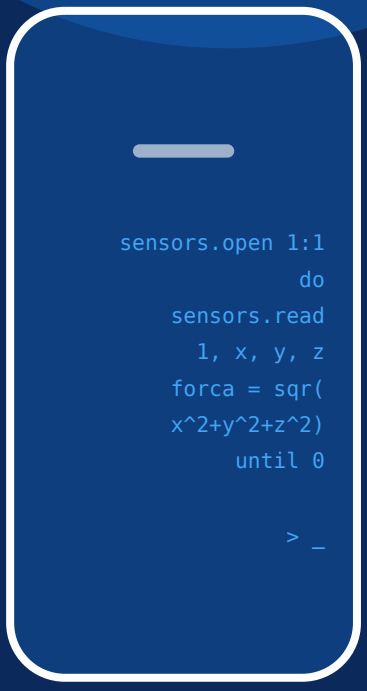
PROGRAMAÇÃO DE BOLSO RFO BASIC

AULA

05

Sensor de nível



Descubra como o celular sabe para onde ele está virado.

RECURSO DO CELULAR

Acelerômetro

repetição · acelerômetro · açúcar sintático

Projeto de Ensino em Programação · Novembro de 2026

NESTA AULA VOCÊ VAI

- ✓ Ler o **acelerômetro** e transformar números em decisões.
- ✓ Criar um detector de chacoalhadas com “memória”.

Aprendemos na aula passada a fazer o celular **repetir**. Hoje, iremos **observar o mundo**, lendo um sensor dezenas de vezes por segundo. Vamos introduzir outro laço de repetição.

1 do ... until

O laço `do ... until` funciona como o laço `while`, mas a condição é verificada no final. Além disso, ele repete o código dentro do laço enquanto a condição é falsa.

Esse laço é ótimo para **validar** respostas, pois pergunta ao menos uma vez e repete até receber algo aceitável:

Listagem 1: Laço `do` / `until`

`laco-do.bas` `basic`

```
1  do
2    input "Digite um número de 1 a 10:", n
3  until n >= 1 & n <= 10
4
5  print "Você escolheu " + int$(n)
```

O código entre `do` e `until` será executado **até** a condição for verdadeira. Você pode ler o código acima como “faça ou execute o código até que a condição seja verdadeira”. No caso concreto acima, enquanto o usuário não digitar um número entre 1 e 10, será solicitado que digite novamente.

ATENÇÃO

Um laço cuja condição nunca muda **nunca termina**. Isso se chama **laço infinito**. Para interromper um programa travado, use o botão voltar do aparelho. Se você deseja construir um laço infinito com *while*, faça `while 1...repeat`, pois o `1` significa que a condição é sempre verdadeira. Outro modo seria `do...until 0`, esse laço só finaliza quando a condição do `until` for verdadeira, mas o `0` é sempre falso.

O próximo exemplo mostra uma nova versão do nosso quiz da tabuada. Primeiro, note a operação `x++`, ela incrementa o valor de `x` em uma unidade. Também temos `x += 1` que significa `x = x + 1`. Chamamos essas formas de açúcar sintático, um modo mais rápido e prático de escrever a mesma coisa.

No nosso código, quando o competidor errar três contas (`erros = 3`), o laço finalizará, pois a condição do `until` se tornará verdadeira. Não precisamos de um acumulador para saber quantas rodadas ocorreram, já que o total será o número de acertos juntamenteo com o de erros.

Listagem 2: Quiz limitado

quiz-do.bas basic

```
1   erros = 0
2   acertos = 0
3   do
4     a = floor(rnd() * 9) + 1
5     b = floor(rnd() * 9) + 1
6     input int$(a) + " x " + int$(b) + " = ", resp
7
8     if resp = a * b
9       acertos += 1
10    else
11      erros++
12    endif
13  until erros = 3
14
15  total = erros + acertos
16  porcentagem = round(acertos / total * 100)
17  print "Você acertou " + int$(acertos) + " de " + int$(total);
18  print " (" + int$(porcentagem) + "%)"
19
20  end
```

Açucar	Significado
`x += y`	`x = x + y`
`x -= y`	`x = x - y`
`x++`	`x = x + 1`
`x--`	`x = x - 1`

ATENÇÃO

Lembre-se que `x = x + y` não é uma equação matemática, mas sim acessar o valor guardado em `x` e somar com o valor em `y`. Depois o resultado é armazenado em `x`, substituindo o antigo. Portanto, devemos ler da direita para a esquerda.

2 O acelerômetro

O acelerômetro mede a **aceleração** do celular, em metros por segundo ao quadrado (m/s²), em três eixos. Mesmo parado, ele sente a **gravidade**, aproximadamente 9,8 m/s², e é isso que revela a posição do aparelho.

Eixo	Direção
`x`	Da esquerda para a direita da tela.
`y`	Da base para o topo da tela.
`z`	Saindo da tela, na direção de quem olha.

Onde a gravidade “aparece” depende de como o celular está:

Posição	x	y	z
De pé (retrato)	0	+9,8	0
De cabeça para baixo	0	−9,8	0
Deitado, tela para cima	0	0	+9,8
Deitado, tela para baixo	0	0	−9,8
Apoiado na lateral esquerda	+9,8	0	0

Os valores reais oscilam um pouco por causa do ruído do sensor e da sua mão trêmula. Por isso trabalharemos com **margens**, como `y > 8`, em vez de `y = 9.8`.

Acessando o sensor

Listagem 3: Acesso ao acelerômetro

`acelerometro.bas` `basic`

```
1   sensors.open 1:1
2   pause 1000
3
4   do
5     sensors.read 1, x, y, z
6     x$ = str$(round(x*10)/10)
7     y$ = str$(round(y*10)/10)
8     z$ = str$(round(z*10)/10)
9
10    print x$, y$, z$
11    pause 500
12  until 0
13
14  sensors.close
15  end
```

`sensors.open 1:1` liga o acelerômetro (o `1` é o código desse sensor). `sensors.read 1, x, y, z` guarda as três medidas nas variáveis `x`, `y` e `z`. A conta `round(x*10)/10` arredonda para uma casa decimal, para a tela ficar limpa.

```
0.1, 9.7, 0.6
0.0, 9.8, 0.4
0.2, 9.8, 0.5
```

RODAR Execute e gire o celular devagar enquanto observa os números. Tente confirmar a tabela de posições.

Outra versão usando a saída formatada, sem arredondamento.

```
x$ = using$("", "%.1f", x)
y$ = using$("", "%.1f", y)
z$ = using$("", "%.1f", z)
```

Determinando a posição do celular

Com os números em mãos, o `if` transforma medidas em significado. Este programa descobre a posição e **só fala quando ela muda**:

Listagem 4: Posição do celular

`posicao.bas` `basic`

```
1   tts.init
2   sensors.open 1
3   pause 1000
4   anterior$ = ""
5   do
6     sensors.read 1, x, y, z
7     if z > 8
8       atual$ = "deitado, tela para cima"
9     elseif z < -8
10      atual$ = "deitado, tela para baixo"
11    elseif y > 8
12      atual$ = "de pé"
13    elseif y < -8
14      atual$ = "de cabeça para baixo"
15    elseif abs(x) > 8
16      atual$ = "de lado"
17    else
18      atual$ = "inclinado"
19    endif
20
21    if atual$ <> anterior$
22      print atual$
23      tts.speak "Estou " + atual$
24      anterior$ = atual$
25    endif
26    pause 500
27  until 0
28
29  sensors.close
30  tts.stop
31  end
```

Repare em três ideias deste programa:

- O `do ... until 0` repete **para sempre**, porque a condição `0` nunca é verdadeira.
- A variável `anterior$` é a **memória** do programa: ele compara o que sente agora com o que sentia antes e só reage a **mudanças**.
- Os `elseif` formam uma **cadeia**, logo os testes são feitos em ordem e o primeiro verdadeiro vence.

RODAR Execute e passeie com o celular pelas posições. Qual posição o programa confundiu? Ajuste os limites.

3 Chacoalhou!

Quando o celular é balançado, as três medidas disparam ao mesmo tempo. Em vez de olhar eixo por eixo, calculamos a **intensidade total** do movimento, que é o tamanho do vetor (x, y, z):

$$\text{força} = \sqrt{x^2 + y^2 + z^2}$$

Parado, essa força vale cerca de 9,8 (só a gravidade), em qualquer posição. Numa chacoalhada firme ela passa de 15. O detector, com memória:

Listagem 5: Chacoalhadas

chacoalho.bas `basic`

```
1   tts.init
2   sensors.open 1
3   pause 1000
4   total = 0
5   chacoalhando = 0
6   do
7     sensors.read 1, x, y, z
8     forca = sqr(x^2 + y^2 + z^2)
9     if forca > 15 & chacoalhando = 0
10      total = total + 1
11      chacoalhando = 1
12      print "Chacoalhadas: " + int$(total)
13      tts.speak int$(total)
14    endif
15    if forca < 11
16      chacoalhando = 0
17    endif
18    pause 50
19  until 0
20
21  sensors.close
22  tts.stop
23  end
```

CONCEITO · VARIÁVEL DE ESTADO

A variável `chacoalhando` guarda se **já estamos no meio** de uma chacoalhada. Sem ela, uma única sacudida duraria várias leituras e seria contada várias vezes. O programa só conta quando o estado muda de “parado” para “chacoalhando”, e só volta a “parado” quando a força cai abaixo de 11.

RODAR Execute e chacoalhe 3 vezes. A contagem fechou em 3? Se não, ajuste os números 15 e 11.

4 Quebre o código!

QUEBRAR Apague a linha `pause 1000` depois de `sensors.open 1:1`. O primeiro valor lido é confiável?

QUEBRAR No contador, apague a linha `chacoalhando = 0` dentro do segundo `if`. O que acontece depois da primeira chacoalhada?

QUEBRAR Troque os limites 15 e 11 por 11 e 15. Por que o programa se comporta de forma estranha?

QUEBRAR Troque `pause 50` por `pause 2000`. Por que as chacoalhadas rápidas passam despercebidas?

QUEBRAR No nível de posição, mova o teste `y > 8` para antes do `z > 8`. Muda alguma coisa? Por quê?

DICA

O terceiro experimento mostra uma ideia profunda de engenharia: usar dois limites diferentes (um para ligar e outro para desligar) evita que o sistema fique "tremendo" na fronteira. Isso se chama **histerese** e está em termostatos, geladeiras e interruptores automáticos.

DESAFIO **Dado chacoalhável**

Crie um programa em que o celular **vira um dado**:

1. a cada chacoalhada, sorteie um número de 1 a 6;
2. **fale** e **escreva** o resultado;
3. se sair 6, diga também "Que sorte!".

Pista: a expressão `floor(rnd() * 6) + 1` sorteia um número inteiro de 1 a 6. Reaproveite o detector de chacoalhadas, pois só a parte dentro do primeiro `if` precisa mudar.

Bônus 1: conte quantas vezes cada lado saiu, em 6 variáveis, e fale o placar quando o usuário virar o celular de cabeça para baixo.

Bônus 2: um "celular dorminhoco": se ele ficar deitado, com `z > 8`, por 5 segundos seguidos, ele fala "Dormi". Pense no acumulador e no que o faz **voltar a zero**.

5 Resumo

Ideia ou comando	Para que serve
`do ... until condição`	Repete até a condição ser verdadeira (ao menos uma vez).

`&` `|`	combinam condições.
`sqr`, `abs`, `mod`, `floor`	Raiz, valor absoluto, resto, arredondar para baixo.
`sensors.open 1:1`	Liga o acelerômetro.
`sensors.read 1, x, y, z`	Lê as três medidas.
Variável de estado	Lembra o que já aconteceu para reagir só a mudanças.

NA PRÓXIMA AULA

Aula 6: Animação. Vamos **desenhar na tela** com os comandos `gr.*` e mover uma bolinha de acordo com a inclinação do celular, numa mistura de acelerômetro, coordenadas e animação. É a porta de entrada para os jogos.